



UNICITY PROTOCOL • SEED PROPOSAL • 2026

SECURING THE AGENTIC ECONOMY.

Proof of identity and permission, bound to the payment itself.

MIKE GAULT • CEO • mike@unity-labs.com
ALAN RADI • COMMERCIAL • alan@unity-labs.com

PREPARED FOR
GREG KIDD @ HARD YAKA

FIRST WE CONVERT STABLECOINS TO OFF-CHAIN BEARER INSTRUMENTS.

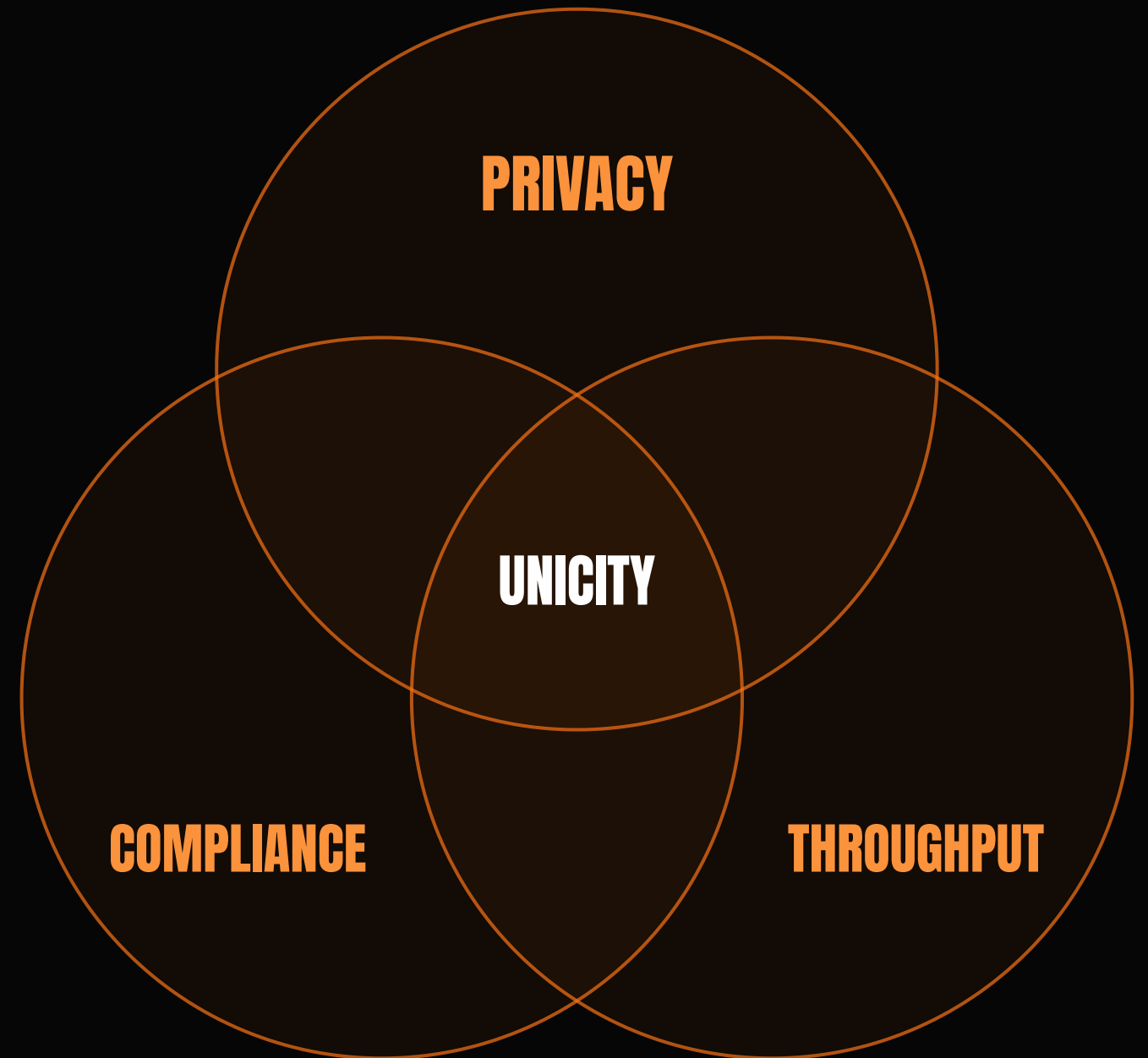
PLASMA USDT



THE TRILEMMA

On a shared ledger every transaction is public – easy to audit, impossible to keep private. Add cryptography (ZK, FHE) to recover privacy and throughput collapses.

Unicity solves this trilemma by introducing bearer instruments.



TOKENS AS BEARER OBJECTS

an asset you **hold**, not a ledger entry the network maintains

SELF-CONTAINED
value lives in the object

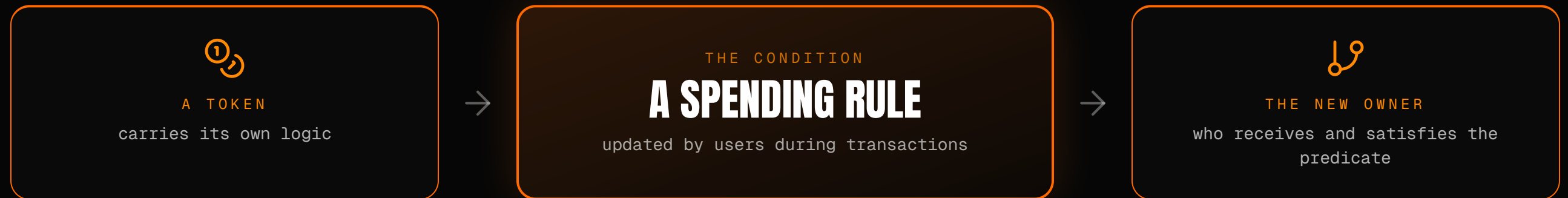
SELF-PROVING
carries its own proof

PEER-TO-PEER
moves with no ledger lookup



EVERY TOKEN IS **PROGRAMMABLE**.

Each token carries its own spending rule – a **predicate** – so transfers can be **conditional**.



A FEW OF THE RULES YOU CAN SET

 **SIGNATURE**
one key signs

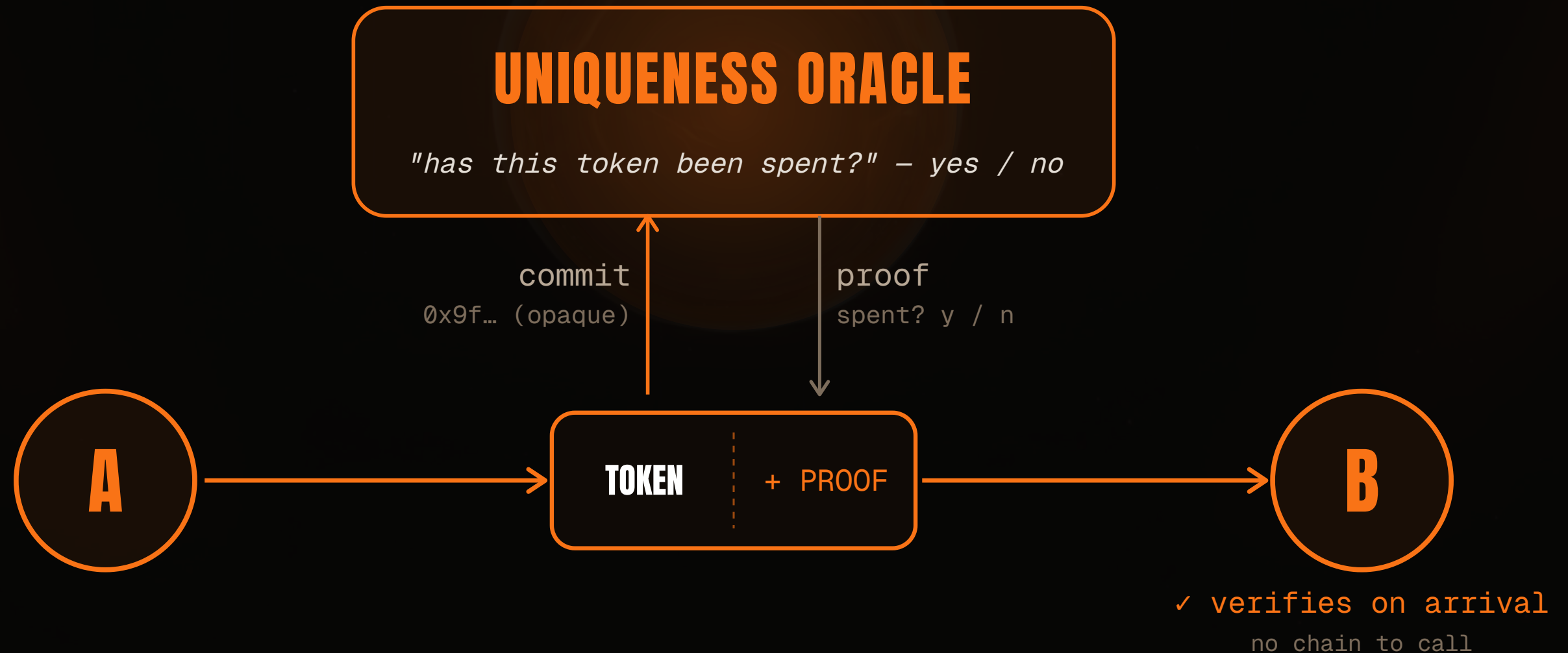
 **MULTI-SIG**
all n keys sign

 **THRESHOLD**
any k-of-n sign

 **TIME-LOCK**
only after τ_0

UNICITY GENERATES PROOFS OF UNIQUENESS (SINGLE SPEND)

Our invention is a **Uniqueness Oracle** – decentralized infrastructure that **proves the absence of double-spend** without seeing transactions.



UNICITY PROVES UNIQUENESS: **NOTHING ELSE**

Proving no double-spend is the Oracle's only job. Transaction validation, storage and P2P communication are all **off-chain** – handled with traditional tools, by the parties who care.

OFF-CHAIN · TRADITIONAL TOOLS

COMMUNICATION

the asset carries its own proof
– the wire just moves bytes

NOSTR

EMAIL

SOCIAL

HOLEPUNCH

STORAGE

state lives with the owner, not
on every node

BROWSER

IPFS

DEVICE

VALIDATION

the recipient verifies – no
validator set in the path

CLIENT-SIDE

IN PARALLEL

🔗 See for yourself – github.com/unicity-sphere/sphere-sdk

THE INEVITABLE UNBUNDLING OF **BLOCKCHAIN.**

Each generation of consensus removed work from the network. We took the unbundling to its logical conclusion.

2009

BITCOIN

Correctness + global ordering.

Every node certifies every transaction. Every node agrees on order.

2023

SUI / FASTPAY

Correctness only.

Validators certify correctness. Ordering removed.

2026

UNICITY

Uniqueness only.

The network attests one thing: has this token been spent? Correctness moves to the edge.

TOKENS ARE NATIVE DATA TYPES, **NOT LEDGER ENTRIES.**

With the ledger gone, the token is a self-contained object – and an object is something an operating system can hold, like a file.

OPERATING SYSTEM

FILE

bytes on disk

PROCESS

running program

SOCKET

network endpoint

TOKEN

value object

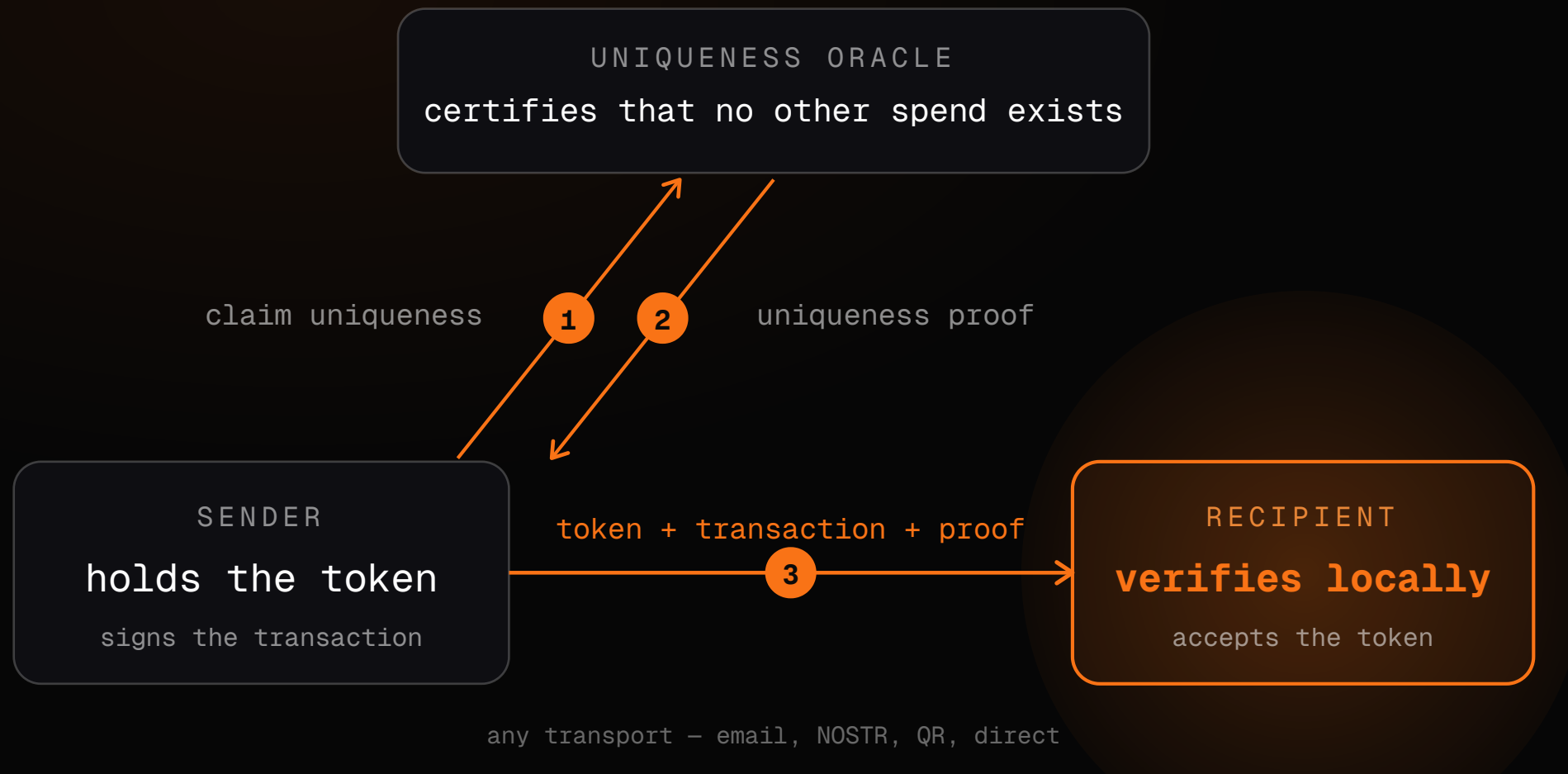
FIVE WINS. **ONE HARD PROBLEM.**

Every architecture has a cost. Ours buys us five things – and
leaves us one hard problem to solve.

SPEED

FAST SETTLEMENT, NO BOTTLENECKS

Tokens move directly between parties **at machine speed**.



THROUGHPUT

The Oracle never validates transactions. Cost per tx collapses; throughput scales horizontally.

30,000 tx/sec

PER SHARD

on a single consumer-class CPU · Plonky3 AIR + Poseidon2

<1 microcent / tx

SUB-CENT · SUB-MILLICENT · SUB-MICROCENT

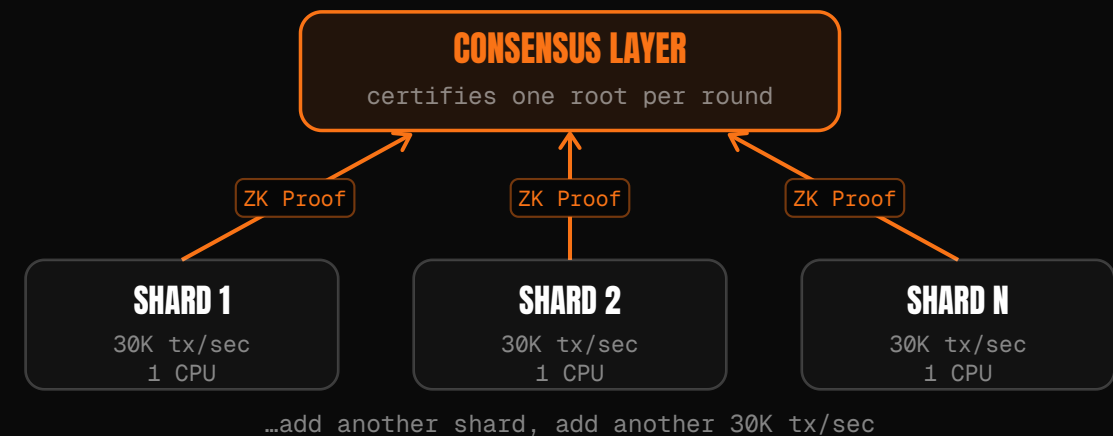
the Oracle certifies one consistency proof per round – not each transaction

∞ shards

HORIZONTAL SCALE

each shard adds another 30K tx/sec · proofs stay succinct

INSIDE THE ORACLE



Why it's cheap. The Oracle never re-executes a transaction – it certifies one prior-state-preservation proof per round: a single SNARK, ~45 ms to verify.

Why it scales. Each shard is independent. No global ordering, no shared validator set, no cross-shard consensus needed.

COMPLIANCE INTEGRATED INTO THE PROTOCOL

TOKENS CAN'T BE RECEIVED WITHOUT THE RIGHT CREDENTIALS.

ENTIRE HISTORY CAN BE VERIFIED COMPLIANT.



RECEIVE PREDICATE

The token's own condition gates receipt – KYC, jurisdiction, accreditation.

PROTOCOL-ENFORCED

Lives in the asset itself, not an app or custodian. No intermediary to bypass.

ISSUER-DEFINED

The issuer sets who may hold it once; the rules travel with the token everywhere.

COMPLIANCE IN THE WALLET, **NOT THE MIDDLE.**

Coinbase wrote the agent-payment standard – but enforces compliance with a facilitator in the middle, watching every transaction.

COINBASE · X402

A MIDDLEMAN IN THE FLOW

- Facilitator sits in the middle of every transaction.
- Compliance enforced by a middleman watching the flow.
- Custodial checkpoints – the network sees your payments.
- Settles in USDC.

UNICITY

COMPLIANT BY CONSTRUCTION

- + Compliance lives inside the wallet – self-custodial.
- + No facilitator, no middleman. Peer-to-peer by default.
- + A non-compliant payment is never even constructed.
- + Settles in USDT.

Same agentic future – but ours is **private, peer-to-peer, and compliant by construction.**

TOTAL PRIVACY BY DEFAULT

THREE OBSERVERS. NONE CAN LEARN ANYTHING.

OBSERVER 01

THE NETWORK

the Unicity service itself

Sees only opaque commitments – never amounts, parties, or balances.

A request is a hash of a state and a hash of transaction data. The nonce that chains one state to the next is hidden by a perfectly hiding commitment – so the service cannot link two consecutive transactions of the same token.

OBSERVER 02

THE SENDER

the person who paid you

Cannot watch when you later spend what they sent.

Once a token reaches you, the previous owner has no way to compute the registry entry your next spend will create. The next state derives from a secret only you hold. Send it, then they're out.

OBSERVER 03

ANYONE WHO KNOWS YOUR ADDRESS

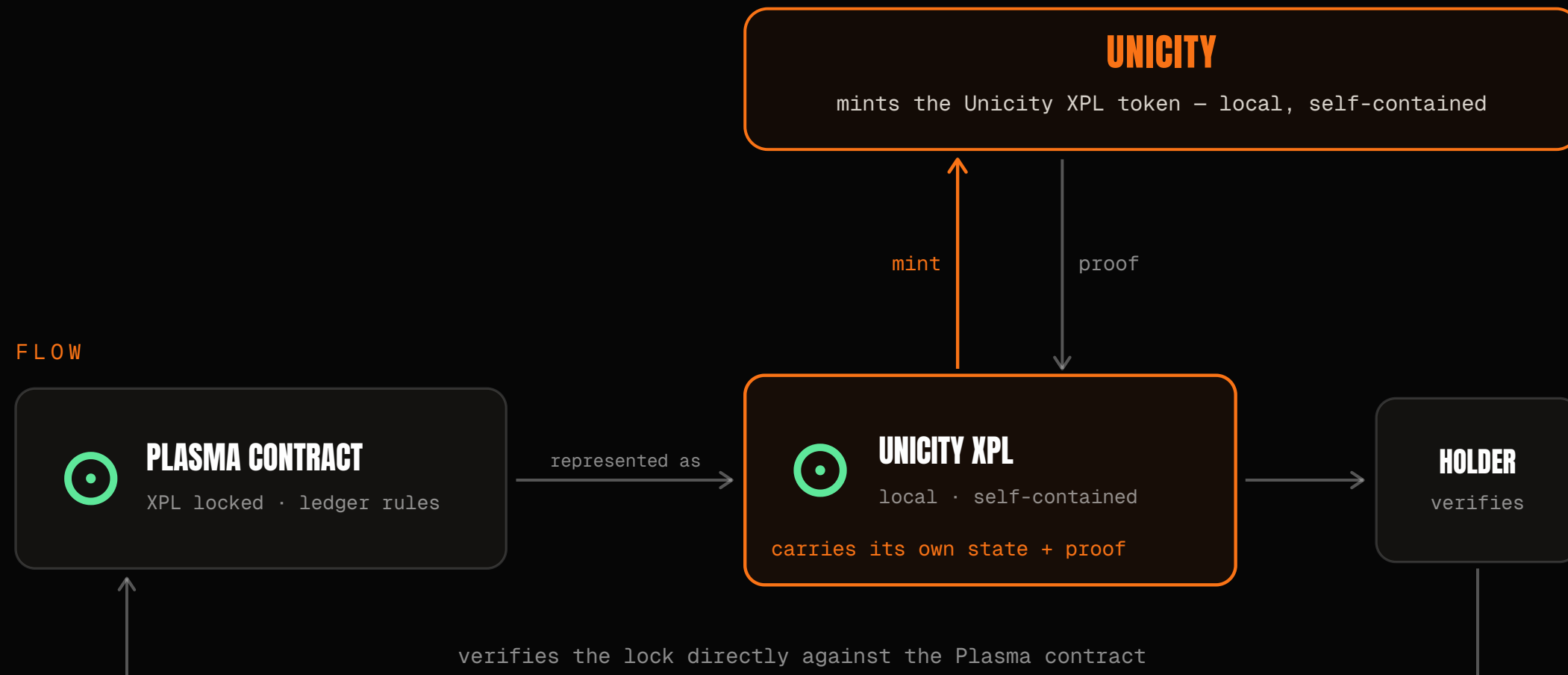
counterparties, partners, ex-employers

Cannot link two payments to the same recipient – even when both went to you.

You publish one persistent public key. Every sender derives a fresh, indistinguishable transaction key against it. No correlation, no UX overhead, no one-time-address management.

BRIDGES ARE ELIMINATED: SOURCE TOKENS ARE VERIFIED DIRECTLY

A Unicity XPL token – verified straight from Plasma. No bridge to trust.



NO BRIDGE · NO CUSTODIAN · NOTHING TO HACK

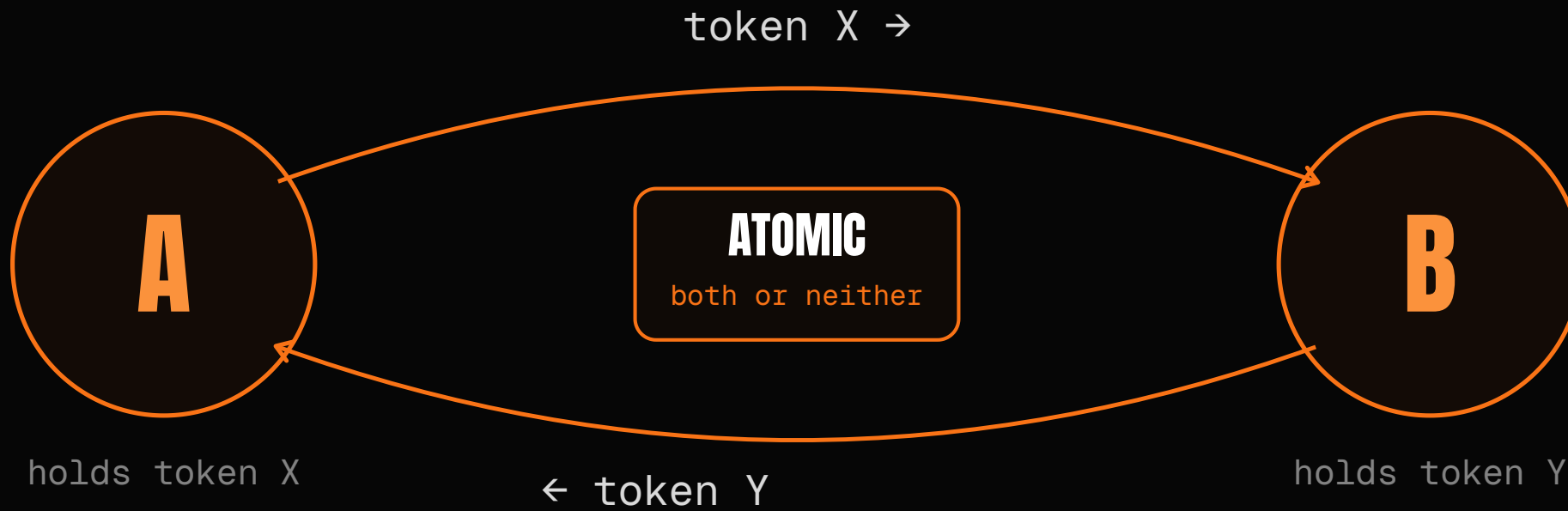
THE HARD PART

ONE HARD PROBLEM: ATOMICITY.

Free in any shared-state model – the database, the global ledger, the world computer all give it by default. Without shared state, "all or nothing" has to be enforced by the tokens themselves.

IN DEFI, THE ATOMIC OPERATION IS THE **SWAP**

Two assets change hands at once – **both legs, or neither.**



Unicity has no shared ledger – so atomicity needs a new paradigm.

that's what predicates solve

THE UNICITY TRUSTLESS ATOMIC SWAP

Both parties commit independently – the swap completes, or everyone keeps their token.

HASHED TIMELOCK (HTLC)

1 · A locks, reveals hash y

2 · B locks against same y

3 · A claims, revealing preimage x

4 · B must claim before timeout

THE TIMING TRAP

B late on step 4 – a dropped connection – and A keeps both.

UNICITY PREDICATE SWAP

A COMMITS

locks their token

B COMMITS

locks their token

UNICITY SERVICE

the shared reference

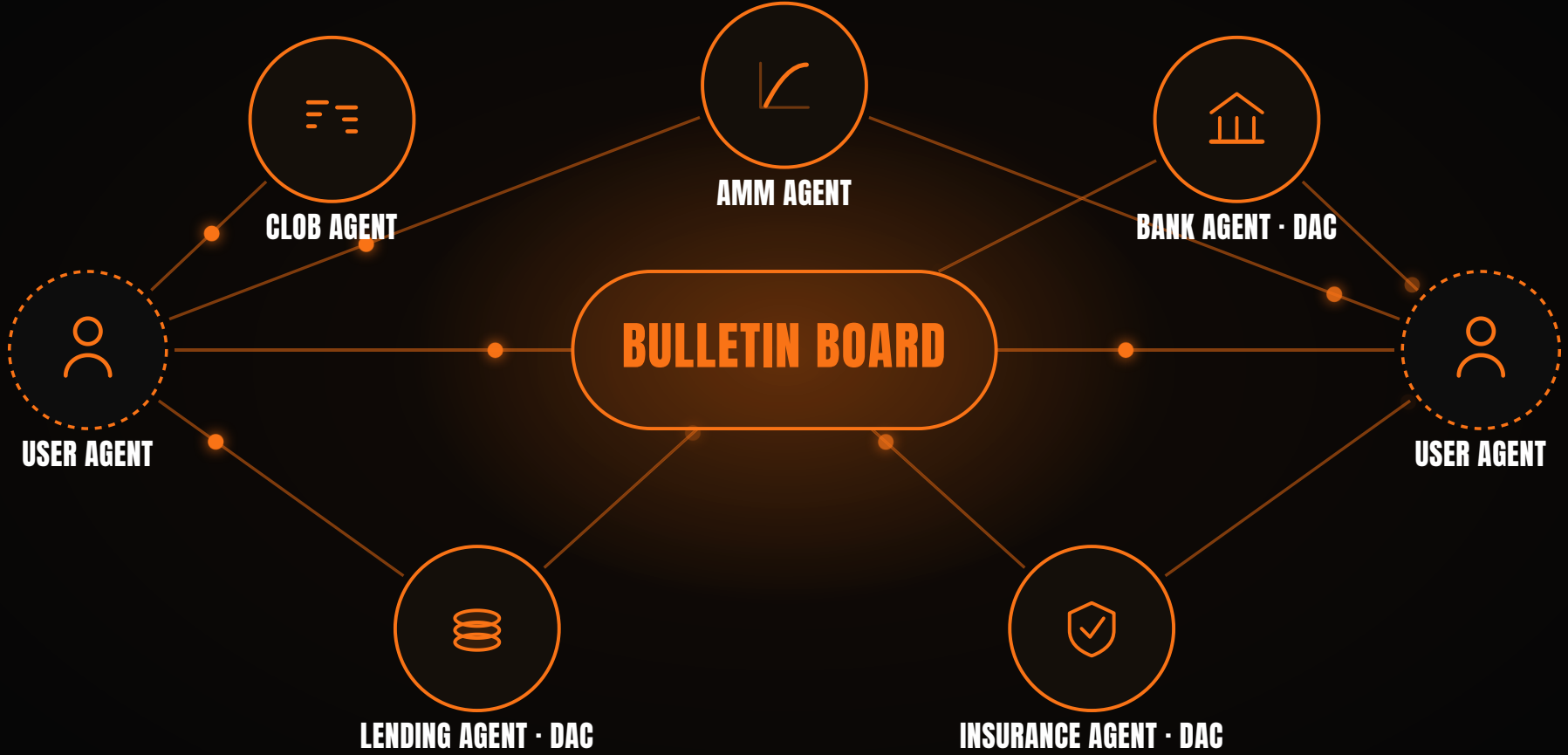
AUTOMATIC FAIRNESS

- ✓ both locked → swap completes
- ✓ either walked away → both refunded
- ✓ no deadlines, no chasing, done

AGENTIC DEFI. AN ECONOMY OF MACHINES.

Frictionless. Massive throughput. Private. Compliant.

A financial system built for machines.

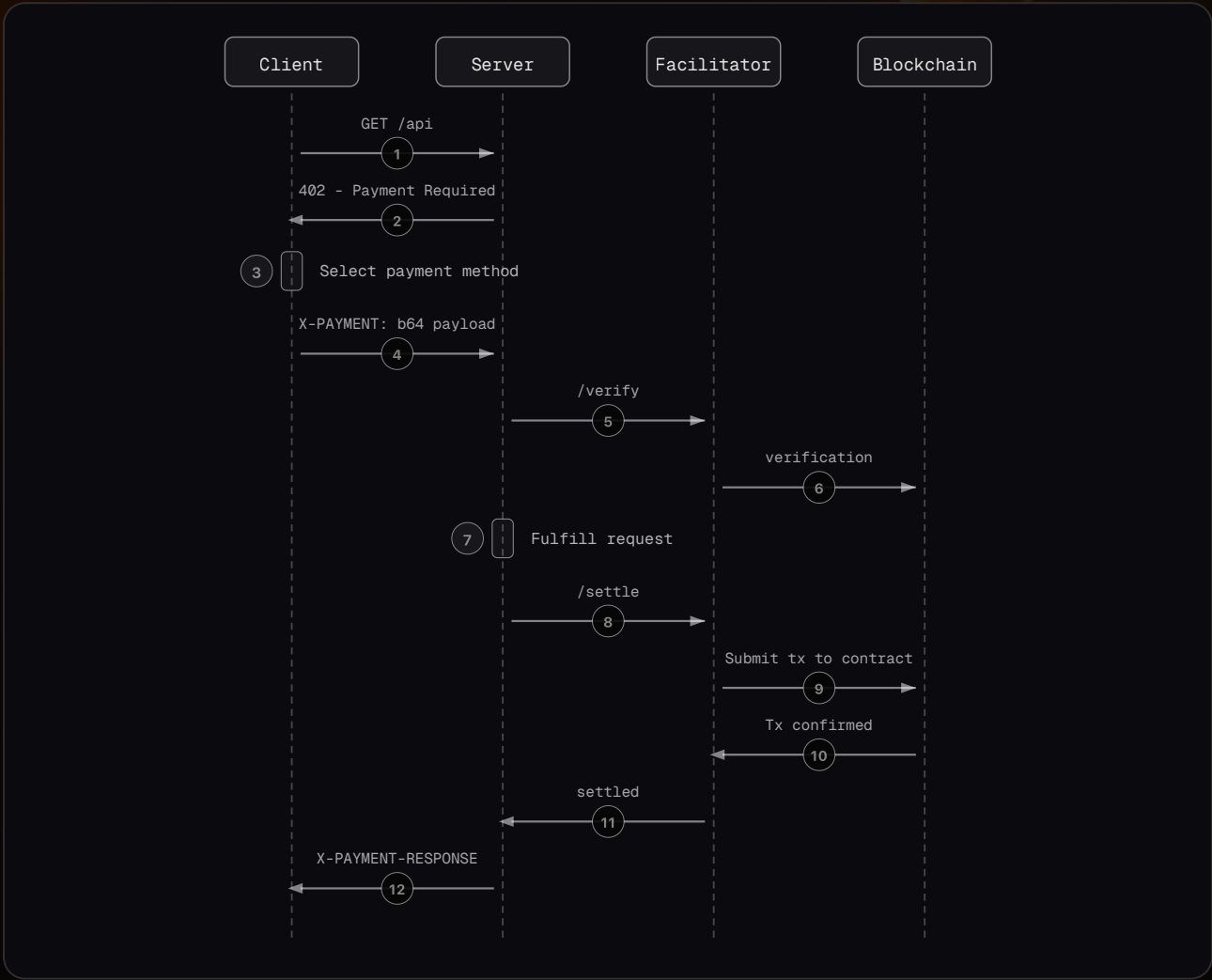


LIVE DEMO: X402 USDT AT 3M TPS WITH PRIVACY AND COMPLIANCE.

x402 Unicity implements and simplifies it – from 12 steps to 5.

[▶ Watch demo – USDT moving A2A at 3M TPS](#)

TRADITIONAL BLOCKCHAIN X402 12 STEPS



UNICITY FRICTIONLESS X402 5 STEPS



CEX SPEED. DEX CUSTODY. PRIVATE AND COMPLIANT.

Binance has the first. Uniswap has the second. Neither has the third. Unicity has all three.

PROPERTY	BINANCE	UNISWAP	HYPERLIQUID	UNICITY CLOB
Match speed	● <1 ms	○ 12 s block	● ~0.2 s	● ~0.2 s
Gas per trade	● \$0	○ \$5-\$50	● \$0	● \$0
Self-custody	○ Exchange-held	● Smart contract	● Non-custodial	● You hold the keys
Privacy	○ Operator sees all	○ Fully public	○ Public positions	● Unlinkable
MEV / sandwich	● Internal front-run	○ Public mempool	● Validator-ordered	● No mempool
Settlement	○ Internal IOU	● On-chain	● Sub-second	● Sub-second
Decentralized	○ Centralized	● Permissionless	○ Admin keys	● Permissionless

THE MATH IS HARDER.
BUT IT'S WORTH IT.

WHAT YOU GET

A NEW FINANCIAL SYSTEM.

Massively parallel. Private. Decentralized.

DON'T TAKE OUR WORD FOR IT — HERE'S THE MATH

Three formal papers (LLM Friendly). Every claim is proven and public.

SYSTEMS ENGINEERING

Unicity Infrastructure: the Aggregation Layer

Risto Laanoja¹
¹Unicity Labs

May 22, 2026

Abstract

Unicity is a novel blockchain protocol with the ambitious goal of enabling token transactions to occur off-chain and, when necessary, on-chain. This paper outlines the infrastructure to guarantee that there are no parallel states of assets, or more specifically, that there is no double-spending, a property we term the *atomicity*. It turns out that the lack of globally shared state and ordering within the blockchain overloads confidentiality. In designing the infrastructure, no compromises were made regarding its trust assumptions. This paper details the design of the Aggregation Layer, the component responsible for producing Proof of Inclusion and Non-inclusion to the user. We analyze its design for efficiency and evaluate the robustness of its trust and security model. We then identify a single critical property that the Consensus Layer must verify on each round—prior to its execution—and observe that the remaining desirable properties of the trustless Key-Value store (consensus free storage, batch incorporation, no placeholder inserts, completeness) are not required by the Unicity Aggregation security model. This optimal statement is implemented [9] as Algebraic Intermediate Representation (AIR) across the top of the StarkEx [14] STARK toolkit. The implementation maintains a proving throughput in excess of 10,000 transactions per second on a single consumer-class CPU, with a transaction proof and term of milliseconds verification time, and no trusted setup.

1

off-chain & offline tokens
ZK proofs · no trusted setup



github.com/unicitynetwork

SECURITY PROOF

The Unicity Execution Layer

Alto Baldea Alto Tunc Risto Laanoja
Vladimir Rogojin

November 2025

Abstract

This paper introduces the Unicity Execution Layer, a modular component of the Unicity framework enabling secure off-chain transactions while maintaining trustless double-spending prevention. We present a formal security model where token ownership is represented by public keys and transfers require digital signatures. We prove three fundamental security properties: (1) no double-spending, each token state can be spent at most once; (2) no blocking—only the legitimate owner can prevent a token from being spent; and (3) service-side privacy—the Unicity Service cannot link transactions with the same token. The service-side privacy is addressed by introducing generalized multipublic-key signatures where each user can generate multiple verifiable public keys, and interaction and non-interactive concrete instantiations, enabling private transactions with verifiable public identity without manual key management overhead.

1 Introduction

Blockchain technology has revolutionized digital asset management by enabling trustless peer-to-peer transactions without relying on centralized authorities. However, traditional blockchain architectures face fundamental scalability limitations that hinder their adoption for high-throughput applications. The core bottleneck stems from the fact that the “security” depends on the number of participating validators, which all have to participate in consensus on ordering, re-executing transactions, and store every produced block.

This paper introduces Unicity, a novel blockchain infrastructure designed to enable secure off-chain transactions while maintaining the trustless guarantees of traditional blockchains. The key insight underlying Unicity is that the vast majority of blockchain operations—transaction execution, smart contract processing, and state transitions—can be moved off-chain, leaving only the essential double-spending prevention mechanism on-chain. This also simplifies on-chain operations, making efficient and self-authenticating implementations possible. By minimizing the data that must be processed by the consensus layer, Unicity achieves linear scalability while preserving the security properties that make blockchains trustworthy. The system consists of three hierarchical layers: the

1

no double-spend · no blocking
service & user-side privacy



github.com/unicitynetwork

PREDICATES & SWAPS

Unicity: Predicates and Atomic Swaps

Alto Baldea Alto Tunc Risto Laanoja
Vladimir Rogojin

November 2025

Abstract

We generalize Unicity token ownership to programmable spending conditions called *predicates*, enabling smart-contracts that functionally executed off-chain directly by relying parties rather than by consensus participants. We prove that the security properties of our Unicity execution layer are preserved under reduction to predicate family satisfiability. To demonstrate the utility of the model, we show how to implement trustless atomic swaps by using predicates.

1 Introduction

Predicates generalize the concepts of token ownership and transfer in the Unicity infrastructure, which in the paper [1] were defined via digital signatures as follows:

- **Owner** – a (legal/physical) person who controls the private key of a digital signature scheme
- **Ownership condition** – the public key pk that corresponds to the private key of the owner.
- **Transfer** – the owner generates a digital signature σ on $m = H(h_m, h_v)$ (where h_m, h_v are the state hash and the transaction hash, respectively) such that $V(pk, m, \sigma) = 1$.

In this paper, we present the following generalized concepts:

- **Owner** – an abstract group of (legal/physical) persons that together control the information necessary to create the next transaction with the token. The information may include private keys.
- **Ownership condition** – a logical condition (predicate) ν .
- **Transfer** – the owner (or a group) presents a bit-string s such that the condition $\nu(r, m, s) = 1$ holds, where $m = H(h_m, h_v)$ and r is the system time (an integer defined by the Unicity service). This means that predicates may also put restrictions on transaction execution time.

1

programmable predicates
trustless atomic swaps



github.com/unicitynetwork

THE PEOPLE

FOUNDERS WHO'VE DONE THIS BEFORE

15 years shipping cryptographic infrastructure to **DARPA**, **NATO**, **Lockheed**.

5 PhD cryptographers in the core team.



Mike Gault

CEO

- PhD Electrical Engineering
- Built & exited Guardtime (ADX:IHC)
- Ex-MD, Barclays Capital



Tony Kenyon

CPTO

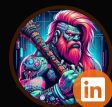
- PhD Machine Learning
- 25 years shipping enterprise AI & infra
- Former Vitruvian (applied ML, security)
- Principal Architect: BT, Nokia, A10



Vladimir Rogojin

ARCHITECTURE

PhD CS · Somnium
Metaverse, Guardtime



Joshua J. Bouw

AI OS

NEAR, Asimov,
Blackcoin · Early
Proof of Stake
pioneer



Matt Law

GROWTH

Outlier Ventures,
Boson, Fetch.ai



Pavel Grigorenko

ENGINEERING

PhD CS · Microsoft,
Guardtime



Jamie Steiner

PRODUCT

JP Morgan

SCIENCE

Ahto Buldas PhD · Ahto Truu PhD · Risto Laanoja PhD · Jian Tan PhD · Dirk Draheim PhD

ENGINEERING

Martti Marran · Igor Mahlinovski · Alexander Khrushkov · Marek Sepp · Lennart Ploom · Dmytro Maryshev · Johannes Ait

ADVISORS

Carlos Domingo (Securitize) · Eric Leandri (Aleria AI) · Arnold Lee (Sphere) · Sylvain LaMarche (Movidone)

SEED ROUND EXTENSION

FROM MAINNET LAUNCH TO INITIAL SCALE

\$3M

USD

STRUCTURE **SAFE + Token Warrants**

TIMELINE **Now**

SEED ROUND BACKERS · SEP 2025

Blockchange Capital Outlier Ventures Tawasal Al Khaleej

WHAT THIS ROUND UNLOCKS

- 01

LAUNCH MAINNET
Network live. Hashlock audit complete. Distribution partners go live.
- 02

CONVERT FIRST DEPLOYMENTS \$20M+ PIPELINE
Sovereign AI Lab and Tier-1 Financial Institution deploy.
Government, Telecom, Banks line up.
- 03

BUILD THE FLYWHEEL NETWORK GROWTH
Developer relations, framework integrations, hackathons. Path to TGE at adoption thresholds.

THIS ROUND FUNDS THE TRANSITION FROM INFRASTRUCTURE → LIVE NETWORK